CS 61A — Week 4

Solutions: Discussion 3, HW3 and assigned real exams

§1 Discussion 3 and related Lecture 10 practice

Q1: Swipe. **Idea.** The first and last output lines are the current last digit. A smaller call produces exactly the lines between them.

```
def swipe(n):
    if n < 10:
        print(n)
    else:
        print(n % 10)
        swipe(n // 10)
        print(n % 10)
```

1. A one-digit input needs one line, so the supplied base prints it once. For a longer input, remove the last digit before recursing.
2. For 2837 the calls first print 7,3,8,2. The call on 2 returns with no second print; the waiting calls then print 8,3,7. Each call returns None.
3. Positive integer division by 10 eventually reaches one digit. The implementation uses no loops, string conversion or indexing.

Q2: Skip Factorial. **Idea.** Remove the first factor n; the remaining product is exactly the skip factorial of n-2.

```
def skip_factorial(n):
    if n <= 2:
        return n
    return n * skip_factorial(n - 2)
```

1. For positive n, the condition $n \leq 2$ covers both final possible factors 1 and 2. Return that factor directly.
2. Otherwise multiply n by the smaller product. Thus 5 gives $5 \cdot 3 \cdot 1 = 15$, and 8 gives $8 \cdot 6 \cdot 4 \cdot 2 = 384$. Decreasing by 2 reaches the base in either parity.

Q3: Is Prime. **Idea.** The original n stays fixed; only the next candidate divisor changes. Retaining n in a parent frame makes one helper argument sufficient.

```
def is_prime(n):
    def f(i):
        """Return whether n has no positive divisor from i through n-1."""
        if i == n:
            return True
        if n % i == 0:
            return False
        return f(i + 1)
    return f(2)
```

1. The helper's promised result is whether none of i through $n-1$ divides n. At i=n there are no candidates left, so return true; n itself is not tested as a disqualifying proper divisor.
2. A divisor at i disproves primality and returns false. If i does not divide n, checking i+1 through $n-1$ is precisely the remaining task.
3. With $n > 1$, the initial candidate 2 is valid and each step decreases the candidates remaining. For 2 the base returns true;16 fails at 2;521 has no candidate divisor and returns true. An external helper would need n as another argument, since it would not have this retained parent binding.

Q4: Recursive Hailstone. **Idea.** Print the current term once, then ask the appropriate helper for this term plus the length of the rest.

```
def hailstone(n):
    print(n)
    if n % 2 == 0:
        return even(n)
    return odd(n)

def even(n):
    return hailstone(n // 2) + 1

def odd(n):
    if n == 1:
        return 1
    return hailstone(n * 3 + 1) + 1
```

1. At 1, hailstone prints 1 and calls odd, which returns 1. This is one element, not zero transitions.
2. An even input continues to n//2; an odd input above 1 continues to 3n+1. A helper adds 1 to that returned term count to include the current printed term.
3. For 10 the call arguments are 10,5,16,8,4,2,1 and the counts on return are 1,2,3,4,5,6,7 in reverse order. The first call returns 7. This proves the count whenever the sequence terminates; it does not prove all positive starts terminate.

Q5: Sevens. **Idea.** The current speaker has already been determined; a number divisible by 7 or containing 7 determines where the next turn goes.

```
def sevens(n, k):
    def f(i, who, direction):
        if i == n:
            return who
        if i % 7 == 0 or has_seven(i):
            direction = -direction
        who = who + direction
        if who > k:
            who = 1
        if who < 1:
            who = k
        return f(i + 1, who, direction)
    return f(1, 1, 1)

def has_seven(n):
    if n == 0:
        return False
    if n % 10 == 7:
        return True
    return has_seven(n // 10)
```

1. The helper invariant is: player who says i, and direction gives the movement order before applying i's rule. At i=n that is the requested answer, so return before another move.
2. Otherwise reverse at a qualifying i, advance once in the resulting direction, wrap 1–k, and recurse on i+1. This preserves the state interpretation and reaches n after finitely many increments.
3. For 5 players, the exact speakers for 1 through 20 are $1, 2, 3, 4, 5, 1, 2, 1, 5, 4, 3, 2, 1, 5, 1, 2, 3, 2, 1, 5$. In particular the source's 2,6,7,8,9,18 cases give 2,1,2,1,5,2. At 7,14 and 17 the direction reverses after that number is said.

Spring 2024 MT1 Q4(e): Streak. **Idea.** A longer streak consists of a shorter valid streak whose last digit agrees with the one removed.

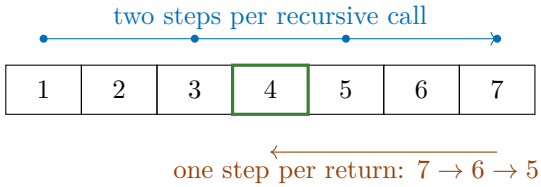
```
def streak(n):
    return (n >= 1 and n <= 6) or (n >= 10 and
        n % 10 == n // 10 % 10 and streak(n // 10))
```

1. The first group accepts the one-digit bases 1–6. For a longer n, compare n%10 with(n//10)%10 and recursively check n//10. These fill the two source blanks.
2. If the recursive prefix is a valid equal-digit dice integer and its last digit matches the removed one, every original digit is equal and valid. Removing digits eventually reaches the one-digit base; a repeated 9 eventually fails there.
3. Thus 22222,4,3333333333333 pass.22322 fails at a digit change;99999 fails its final 9 base;505,707,7070 fail either digit agreement or the invalid one-digit base. All actual source examples are covered.

Q6 (optional): Karel. **Idea.** A smaller call advances through the remaining row and returns facing west; each waiting call then needs only one move back.

```
def main():
    if front_is_clear():
        move()
        if front_is_clear():
            move()
            main()
            move()
        else:
            turn_left()
            turn_left()
    else:
        turn_left()
        turn_left()
```

1. A call starts facing east with L squares left including its current square. If L=1, turn west and stop. If L=2, move to the last square, turn west and stop. These are the two base branches.
2. For L ≥ 3, advance two squares and solve the same task on L -2 squares. By its contract, the smaller call finishes facing west at offset $\lfloor (L-2)/2 \rfloor$ from its own start. One more move west gives offset $2 + \lfloor (L-2)/2 \rfloor - 1 = \lfloor L/2 \rfloor$ from the original start.
3. Thus the final column is $1 + \lfloor n/2 \rfloor$:4 for n = 7, and the right of the two middle columns for even n. Every forward move is guarded by a clear test. The return move is safe because its smaller call started at least two squares east of the waiting call.
4. All branches end facing west. The function uses only conditionals and calls in its body, including its recursive call; it never reads or stores n. The linked UI starts bottom-left facing east; the proof uses that initial heading.



For the actual 7-square example, the calls begin at columns 1,3,5,7. Turning west at the end and taking one return move per waiting call reaches column 4.

§2 Homework 3

Q1: Num Eights. **Idea.** Use the last digit to decide whether to add1 to the count on the shorter prefix. No variable assignment is needed.

```
def num_eights(n):
    if n < 10:
        return 1 if n == 8 else 0
    if n % 10 == 8:
        return 1 + num_eights(n // 10)
    return num_eights(n // 10)
```

1. A one-digit number contributes 1 precisely when it is 8. A longer number contributes the shorter prefix's count, plus 1 only when its last digit is 8.
2. For 8782089, the successive tested digits are 9,8,0,2,8,7,8; three contribute 1. This gives 3. The source's remaining inputs 3,8,88888888,2638,86380,12345 give 0,1,8,1,2,0.
3. Removing a digit reaches a one-digit base. The entire function contains no assignment or loop statement; its branches call the same recursive count.

Q2: Digit Distance. **Idea.** Removing the last digit removes exactly the final adjacent pair; all earlier pairs remain unchanged.

```
def digit_distance(n):
    if n < 10:
        return 0
    return abs(n % 10 - n // 10 % 10) + digit_distance(n // 10)
```

1. A single digit has no pair and contributes 0. Otherwise add the absolute difference between the last two digits to the distance of n//10.
2. For 71253, the right-to-left pair contributions are 2,3,1,6, with sum 12. For 314 they are 3 and 2, with sum 5;777 gives 0. The longer source cases 31415926535 and 3464660003 give 32 and 16.
3. Integer division removes one digit on each call; no pair is counted twice or omitted, and there are no loops.

Q3: Interleaved Sum. **Idea.** Alternate the function being called instead of testing an index's parity.

```
def interleaved_sum(n, f_odd, f_even):
    def odd(k):
        if k > n:
            return 0
        return f_odd(k) + even(k + 1)
    def even(k):
        if k > n:
            return 0
        return f_even(k) + odd(k + 1)
    return odd(1)
```

1. odd(k) is called only at odd k; it applies f_odd there and asks even for k+1. The even helper does the corresponding even step and asks odd for the next index.
2. Above n, no terms remain and the result is 0. Increasing k terminates, and the alternating calls include each index 1 through n once.
3. With identity/square and n = 5, the sum is $1 + 4 + 3 + 16 + 5 = 29$. Reversing them gives $1 + 2 + 9 + 4 + 25 = 41$. At n = 4, triple/square gives $3 + 4 + 9 + 16 = 32$; square/triple gives $1 + 6 + 9 + 12 = 28$.
4. The implementation uses addition and comparison only to advance/control indices. It contains none of the operators or loops banned by the public tests; supplied term functions may perform their own arithmetic.

Q4: Count Coins. **Idea.** At each denomination, separate combinations that use at least one such coin from combinations that use none.

```
def next_smaller_coin(coin):
    if coin == 25:
        return 10
    if coin == 10:
        return 5
    if coin == 5:
        return 1

def count_coins(total):
    def count(remaining, coin):
        if remaining == 0:
            return 1
        if remaining < 0 or coin is None:
            return 0
        return (count(remaining - coin, coin) +
                count(remaining, next_smaller_coin(coin)))
    return count(total, 25)
```

1. The supplied next_smaller_coin returns 10,5,1 and then None. The helper's result counts combinations totaling remaining with the current coin and smaller denominations.
2. Using a current coin leaves remaining − coin and permits more copies of that coin. Using none leaves remaining unchanged and lowers the denomination. Those cases are disjoint and exhaustive, so add their counts.
3. At remaining 0, the coins already selected complete one combination. Negative remaining is impossible; a positive remaining with coin None is impossible. The success test comes first, so completing a total counts even when no denominations remain.
4. For 15, the six combinations are exactly those listed in the problem. The source totals 10,20,45,100,200 give 4,9,39,242,1463. Every recursive branch either lowers the positive remaining amount or advances along a finite denomination list; it terminates with no loops.

Q5: Maximum Subsequence. **Idea.** Every candidate either uses the current last digit or omits it. Find the best candidate in each group, then compare.

```
def max_subseq(n, t):
    if n == 0 or t == 0:
        return 0
    with_last = max_subseq(n // 10, t - 1) * 10 + n % 10
    without_last = max_subseq(n // 10, t)
    return max(with_last, without_last)
```

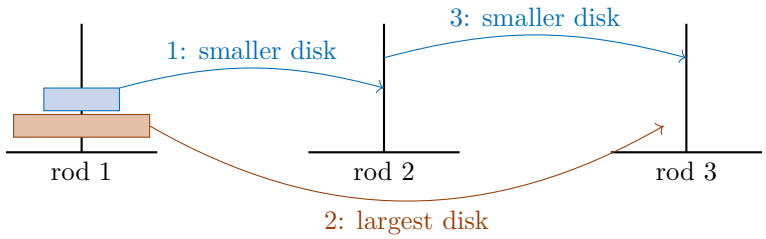
1. If no digits or no slots remain, the best numeric result is 0. Otherwise keeping the last digit uses a slot and appends it to the best preceding result; skipping it leaves the slot limit unchanged.
2. Multiplying by 10 and adding a fixed digit preserves numerical order. Therefore the best preceding subsequence also gives the best candidate that keeps this last digit. Taking the maximum covers both exhaustive groups.
3. For $n = 2012, t = 2$, the keep-last branch gives $2 \cdot 10 + 2 = 22$ and the skip-last branch gives 21; return 22. For $n = 20125, t = 3$, keeping 5 gives 225 while skipping it gives 212.
4. The source's other cases give 20125 at limits 5 or 6,345 at 12345/limit 3,0 at limit 0, and 5 at limit 1. Every call removes a digit; the implementation uses recursion and no loops.

Q6 (just for fun): Towers of Hanoi. **Idea.** The largest disk cannot move until the smaller stack is off it, and its destination must also be free of smaller disks. With three rods, the spare rod is forced.

```
def print_move(origin, destination):
    print('Move the top disk from rod', origin, 'to rod', destination)

def move_stack(n, start, end):
    assert 1 <= start <= 3 and 1 <= end <= 3 and start != end, 'Bad start/end'
    if n == 1:
        print_move(start, end)
    else:
        spare = 6 - start - end
        move_stack(n - 1, start, spare)
        print_move(start, end)
        move_stack(n - 1, spare, end)
```

1. One disk needs exactly the supplied print_move. For n $>$ 1, move the top $n-1$ disks to the spare rod. The recursive contract preserves their legal order.
2. The largest disk is now exposed and the destination has no smaller disk, so move it. Transfer the smaller stack from spare to destination using the same recursive contract; all disks are now in their required order.
3. The three rod labels sum to 6, so the spare rod is 6 − start − end. Each recursive stack has fewer disks and eventually reaches one disk. Larger disks below these subproblems do not obstruct their smaller disks.
4. For n = 2,start 1,end 3, the moves are 1 →2,1 →3,2 →3. For n = 3 the exact source sequence is 1 →3,1 →2,3 →2,1 →3,2 →1,2 →3,1 →3. Each is printed using the given wording.



The actual 2-disk case: move the smaller disk aside before the largest can move, then place it back above the largest. For n disks, replace the smaller disk by the ordered top n-1 stack.

Q7 (just for fun): Anonymous Factorial. **Idea.** A function parameter can supply the self-reference that a global def-name would usually supply.

```
from operator import sub, mul

def make_anonymous_factorial():
    return (lambda f: lambda n: f(f, n))(
        lambda f, n: 1 if n == 0 else mul(n, f(f, sub(n, 1))))
```

1. The first lambda receives a function f and returns a function of n calling f(f,n). The argument supplied for f is the second lambda, which accepts its own function value as its first parameter.
2. Its recursive branch uses f(f,$n-1$), passing that same value again. Thus every recursive application has access to itself without a global recursive name. At 0 it returns 1; at positive n it multiplies n by the result at $n-1$.
3. The resulting function at 5 returns 120. Its return expression uses only lambdas, calls and a conditional expression, with no reference to make_anonymous_factorial. The imports precede their use. The original source's named lambda example instead refers to global fact; this expression removes that dependency.

§3 Assigned real exams

Fall 2017 MT1 Q4(a): Collapse. **Idea.** If the last two digits agree, discard the last copy; otherwise preserve the last digit after collapsing the earlier digits.

```
def collapse(n):
    left, last = n // 10, n % 10
    if left == 0:
        return last
    elif left % 10 == last:
        return collapse(left)
    else:
        return collapse(left) * 10 + last
```

1. left = 0 means n has one digit, including n = 0; return last. Otherwise left's last digit is the digit immediately to last's left.
2. If these agree, removing the current last digit leaves one fewer copy of the same consecutive run. If they differ, the collapsed prefix ends in left's last digit and can safely have last appended without creating a duplicate boundary.
3. For 12234441, collapsing each run gives 1,2,3,4,1 and result 12341. For 11200000013333 it gives 1,2,0,1,3 and 12013. Inputs 1234,0,3 remain unchanged. Each call strips a digit and preserves all nonduplicate transitions.

Fall 2019 Final Q6(b): Contains. **Idea.** For the requested last digit, choose the rightmost matching digit available in b. This leaves as much of b's prefix as possible for all earlier requested digits.

```
def contains(a, b):
    if a == b:
        return True
    if a > b:
        return False
    if a % 10 == b % 10:
        return contains(a // 10, b // 10)
    else:
        return contains(a, b // 10)
```

1. Equal a,b succeeds immediately. A decimal subsequence cannot have a greater numeric value than its source: using fewer digits gives a shorter value, while using all digits leaves the same number. Thus a > b fails.
2. If last digits agree, match them and remove both. Any successful placement of a's last digit could be shifted right to this last position, leaving all its earlier matches available. This justifies choosing the matching last digit without another branch.
3. If last digits differ, b's last digit cannot match the last requested digit and must be discarded. Keep a while shortening b. Every call shortens b until success or failure is decided.
4. For 357 in 12345678, skip 8, match 7, skip 6, match 5, skip 4, match 3, then no requested digits remain: true. For 753 the requested order fails;357 in 37 lacks 5 and fails.
5. The official convention represents an empty remainder by a = 0, even on initial input. Hence contains(0,7) is true under that convention, not a claim that the ordinary one-digit numeral 0 occurs in 7. The implementation uses none of the forbidden conversions, containers or indexing syntax.